

Parallel Processing and Pipeline Hazards

Lecture Notes — Week 11

PCC CS-401: Computer Organization and Architecture

September 5, 2026

11.1 From Latency Hiding to Instruction-Level Parallelism

Week 9 used locality to hide DRAM latency behind cache, and Week 10 used pages, frames, and storage to give a process a larger, protected address space than physical memory alone could provide. Both mechanisms ask the same question: how can the machine keep useful work moving while one operation is slow? This week asks the question at the processor itself. If several useful operations are ready at once, the processor can overlap them, but only when the required resources and data dependencies permit the overlap.

The running instruction stream for this chapter is

```
LW R1,0(R2); ADD R3,R1,R4; SUB R5,R3,R6; BEQ R5,R0,L; OR R7,R8,R9.
```

It will let us identify dependencies and then ask which mechanism pays for each one. The chapter proceeds in four connected topics: classifying the kind of parallel work, constructing a five-stage pipeline, diagnosing hazards, and choosing a resolution whose throughput gain justifies its cost.

11.2 Flynn’s Classification and Scales of Parallelism

11.2.1 Instruction and Data Streams

The first question is not “how many ALUs are present?” It is “how many instruction streams and data streams are moving?” An instruction stream is the sequence of operations being issued. A data stream is the sequence of operands or data elements consumed by those operations. Flynn’s taxonomy classifies a computer by the number of concurrent streams; it is a taxonomy of organization, not a ranking from bad to good. This is the standard course treatment in Hamacher, Ch. 5 [1], while Stallings gives a brief parallel-organization discussion in S20.1 [2].

Definition (Flynn’s taxonomy). *Flynn’s taxonomy classifies an organization by its number of instruction streams and data streams. The four labels are SISD (single instruction, single data), SIMD (single instruction, multiple data), MISD (multiple instruction, single data), and MIMD (multiple instruction, multiple data).*

The four classes can be read directly from the following table.

Class	Instruction streams	Data streams	Typical interpretation
SISD	One	One	Conventional sequential core; one instruction acts on one data item
SIMD	One	Many	One operation applied across a vector or image row
MISD	Many	One	Rare and specialized; multiple operations inspect one stream
MIMD	Many	Many	Multicore, multiprocessor, or cluster with independent work

One physical chip can support more than one mode at different levels. For example, multiple cores may each run an SISD-like stream while a vector unit executes SIMD operations. Therefore the classification should be tied to the level being described, not assigned once to the whole chip without saying which organization is under discussion.

Example 11.2.1 (Classifying three workloads). *Consider the following workloads.*

1. Add two 1024-element vectors using one vector instruction per chunk.
2. Run four independent matrix tiles on four cores.
3. Apply several independent filters to the same sensor stream.

The first workload is SIMD: one operation pattern is applied to many data elements. The second is MIMD: the cores execute independent instruction streams over independent data regions. The third resembles MISD under a strict stream definition because several operations inspect one input stream, but in practice it is often implemented as MIMD workers sharing an input buffer. The taxonomy describes the organization; the workload determines whether that organization is useful.

11.2.2 DLP, TLP, and ILP

Flynn's classes describe streams, while data-level, thread-level, and instruction-level parallelism describe where useful concurrency comes from.

Definition (Data-level parallelism (DLP)). *Data-level parallelism applies the same operation to many elements, as in vector addition or image pixels.*

Definition (Thread-level parallelism (TLP)). *Thread-level parallelism executes independent instruction sequences on separate cores or hardware threads.*

Definition (Instruction-level parallelism (ILP)). *Instruction-level parallelism exists when one instruction stream contains independent instructions that a processor can overlap.*

These are different descriptions. A processor with four cores, each running a different loop iteration, has TLP because four independent instruction streams execute on separate cores. If each core also contains a five-stage pipeline, it has ILP because each core overlaps instructions within its own stream. It is not automatically SIMD: the cores may execute different instructions or different control paths. A good classification names the level and the evidence rather than counting only the number of cores.

The rest of this chapter focuses on ILP inside one instruction stream. That is where hazards become the price of overlap: an instruction may be independent enough to share the machine with another instruction, or it may need a resource or value that is not ready yet.

11.3 The Five-Stage Pipeline Model

11.3.1 Why Overlap Helps

Suppose every instruction needs five stages: fetch, decode, execute, memory, and writeback. If one instruction occupies the whole path before the next starts, four stage resources sit idle for most of the time. A **pipeline** places pipeline registers between stages and overlaps different instructions in different stages. After the pipeline fills, one completed instruction can emerge each cycle even though each individual instruction still requires all five stages. The promise is throughput, not a magical reduction of every instruction's latency. This is the standard pipeline motivation in Hamacher, Ch. 10 [1], and Stallings 11e, S16.4, PDF pp. 667–685 [2].

Definition (Five-stage pipeline). *The teaching model divides an instruction path into IF (instruction fetch), ID (decode and register read), EX (ALU operation or branch work), MEM (data memory access), and WB (register write). The same instruction visits all five stages, while different instructions occupy different stages simultaneously.*

Each stage advances on a clock edge. Pipeline registers hold the intermediate results and control information needed by the next stage. The slowest stage, plus the pipeline-register overhead, constrains the clock period. Real processors may split, merge, or repeat these functions; the five-stage model is a deliberately useful abstraction for reasoning about timing and hazards.

11.3.2 Filling the Pipeline

Example 11.3.1 (Four independent instructions). *Assume four independent instructions and one cycle per stage. With no hazards, the schedule is:*

	1	2	3	4	5	6	7	8
I_1	IF	ID	EX	MEM	WB			
I_2		IF	ID	EX	MEM	WB		
I_3			IF	ID	EX	MEM	WB	
I_4				IF	ID	EX	MEM	WB

Without overlap, each instruction occupies five stage-cycles, so the total is $4 \times 5 = 20$ stage-cycles. With overlap, the first instruction takes five cycles to emerge and each of the remaining three takes one additional cycle: $5 + (4 - 1) = 8$ cycles. The finite-stream speedup is therefore

$$\frac{20}{8} = 2.5.$$

It is below the ideal five because the stream must first fill the pipeline and then drain it. The timing table makes both effects visible: cycle 1 contains only I_1 's IF stage, and after I_4 enters, the already-started instructions still need to finish.

11.3.3 Latency, Throughput, and Speedup

Three quantities answer three different questions. **Latency** is the time from an instruction's IF to its WB. **Throughput** is how often completed instructions emerge. **Speedup** compares total

completion time with a non-overlapped baseline. Confusing latency with throughput produces the false claim that a five-stage pipeline makes every instruction take one cycle.

With k balanced stages and n independent instructions, the idealized pipelined time is

$$T_{\text{pipe}} \approx k + (n - 1) \text{ cycles,}$$

whereas sequential time is nk cycles. For large n , ideal speedup approaches k , provided stages are balanced and hazards, fill, drain, and pipeline-register overhead are negligible. An individual instruction still traverses all k stages, so its latency remains about k cycles in this model. The valid claim is that, after fill, independent instructions can complete at a rate of one per cycle.

Stalls and flushes reduce achieved throughput. A stall appears as a bubble in the timing chart; a flush appears as work that was fetched but is discarded. A deeper pipeline can raise clock frequency by shortening individual stages, but it can also increase branch penalty, register overhead, and recovery cost.

11.3.4 Questions That Shape a Pipeline

Pipeline design is an architectural choice because it determines CPI, area, power, and verification burden. The designer must ask:

1. **Stage balance:** can work be divided so no stage dominates the clock period?
2. **Pipeline registers:** what state must cross each boundary, and what setup and clock-to-q overhead is added?
3. **Resource replication:** can instruction fetch and data access happen together, or must they share one memory port?
4. **Dependence handling:** where can a value be forwarded, and when must the pipeline wait?
5. **Control recovery:** how many younger stages must be flushed after a branch is resolved?

11.4 Hazard Taxonomy

Definition (Pipeline hazard). *A pipeline hazard is a condition that prevents the next instruction from executing in its intended cycle. The remedy must match the cause: add or duplicate a resource, move or forward data, or predict and recover from control flow.*

There are three principal categories. A **structural hazard** occurs when two stages need the same hardware at once. A **data hazard** occurs when an instruction depends on a value whose production or writeback has not completed. A **control hazard** occurs when the next instruction address is uncertain, usually because of a branch or exception.

11.4.1 Structural Hazards

In a five-stage pipeline, suppose I_1 is in MEM while I_2 is in IF. If one single-ported memory serves both instruction fetches and data accesses, both stages request the same port in the same cycle. The problem is resource demand, not a relationship between the values computed by the instructions.

There are three direct responses:

1. Stall one instruction, inserting a bubble.

2. Split instruction and data memories or caches so the accesses proceed in parallel.
3. Use multiplexed or time-multiplexed hardware, paying an area, energy, or timing cost.

11.4.2 Data Hazards and Dependence Names

The running stream begins:

```
LW R1,0(R2)
ADD R3,R1,R4
SUB R5,R3,R6
```

The ADD needs the value produced by the LW, so the pair has a read-after-write (RAW) dependence. The SUB needs the value produced by the ADD, so a second RAW dependence chains the stream. These are true dependences: the consumer really needs the producer's value.

Write-after-read (WAR) and write-after-write (WAW) hazards can arise when instructions execute or complete out of order. In a simple in-order five-stage pipeline, RAW is the central hazard because the normal stage order already limits the false name conflicts.

Example 11.4.1 (A load-use stall). *Assume a load's data becomes available after MEM, while the dependent ALU instruction needs it at the start of EX. Even with forwarding, the value is available one stage too late for the immediately following ADD. The schedule therefore contains one bubble:*

	1	2	3	4	5	6	7	8
LW	IF	ID	EX	MEM	WB			
ADD		IF	ID	stall	EX	MEM	WB	
SUB			IF	stall	ID	EX	MEM	WB

The hazard unit freezes the PC and the IF/ID pipeline register for one cycle, while inserting a bubble into EX by clearing the control fields entering that stage. After the wait, the load result can be forwarded from the MEM/WB boundary to the dependent ADD's EX input. The stall is not a failure of forwarding; it is the one cycle needed because memory data arrives too late.

11.4.3 Control Hazards

A branch changes the instruction stream. The fetch unit may fetch a fall-through instruction before the branch condition and target are known. If the branch is taken, those instructions are wrong-path work and must be prevented from changing architectural state. The penalty is the number of younger stages that must be flushed or redirected when the branch resolves.

The designer can delay the decision, which is simple but inserts bubbles; can predict and recover, which keeps fetching but flushes on a misprediction; or can move branch resolution earlier, which reduces the penalty but lengthens an earlier stage and may add hardware to the critical path.

Example 11.4.2 (Diagnosing three hazards). *1. If one memory port is requested by IF and MEM in the same cycle, the hazard is structural. Duplicate, split, or schedule the resource.*

2. If an ADD reads a register before a preceding load produces it, the hazard is data/RAW. Forward when possible and otherwise stall.

3. If a conditional branch has not resolved its target, the hazard is control. Predict, delay, or flush wrong-path work.

In each case, a complete answer gives both the category and the mechanism that pays for avoiding it.

11.5 Resolving Hazards and Spending the Throughput Budget

11.5.1 Stalls and Bubbles

When the next instruction cannot safely advance, the hazard unit holds the blocked instructions and inserts a no-op bubble into the following stage. The program's meaning is preserved because the dependent operation waits for the required resource or value. A stall can freeze the PC, freeze an inter-stage register, and clear the control fields entering the next stage.

The cost is extra cycles. If a fraction s of instructions incurs an average of b bubbles, then, before other effects are included,

$$\text{CPI} = 1 + s \times b.$$

Thus a hazard remedy should be evaluated by the useful work it preserves and the cycles it adds, not merely by whether it keeps a stage occupied.

11.5.2 Forwarding and Bypassing

Suppose an ALU result is already at the EX/MEM boundary. The next ALU operation can consume that result directly instead of waiting for WB to write the register file and ID to read it. Comparators detect whether a source register matches a destination register in a later stage, and multiplexers select the newest available value for each ALU input. Forwarding or data bypassing removes many ALU-to-ALU RAW stalls. It cannot remove a load-use delay when memory data arrives too late. Forwarding is part of the indexed pipeline treatment in Stallings 11e, S16.4, and its RISC discussion is cited in the deck with the same source [2].

11.5.3 Scheduling, Renaming, and Readiness Tracking

The compiler can move an independent instruction into a delay slot or between a producer and consumer when the ISA permits it. Register renaming gives independent writes different physical destinations, removing false WAR and WAW name conflicts. Scoreboarding and reservation stations track operand readiness and resource availability so independent instructions can proceed.

These techniques expose more ILP, but require metadata, selection logic, recovery rules, and stronger verification. No scheduling trick can remove a true RAW dependence: the consumer still needs the producer's value.

11.5.4 Prediction and Recovery

A branch predictor guesses direction and sometimes the target so that fetch continues without waiting for full branch resolution. A correct prediction turns a control hazard into useful work. A wrong prediction requires a **flush**: younger instructions are invalidated and fetch restarts from the correct target. The expected branch cost is approximately

$$\Delta\text{CPI} \approx f_{br} p_{miss} P_{flush},$$

where f_{br} is branch frequency, p_{miss} is the misprediction rate, and P_{flush} is the recovery penalty. Deeper pipelines can increase P_{flush} even when their clock is faster.

11.5.5 A Design Comparison

Every architecture choice spends some part of a throughput budget.

Choice	Potential gain	New cost	Question to ask
Deeper pipeline	Shorter stage delay; higher clock	More registers, bubbles, branch penalty	Are hazards predictable enough?
Duplicate resources	Fewer structural conflicts	Area, power, coherence/consistency	Is the conflict frequent?
Forwarding network	Fewer RAW stalls	Comparators, multiplexers, timing paths	Can the value arrive in time?
Branch prediction	Higher control-flow throughput	Predictor state and flush recovery	What is the miss penalty?
Out-of-order issue	More ILP exposed	Rename, queues, precise recovery	Is added complexity worth it?

The design principle is to maximize useful completed work per unit time, not merely the clock rate or the number of stages.

Example 11.5.1 (A complete diagnosis of the running stream). *Consider again:*

LW R1,0(R2); ADD R3,R1,R4; SUB R5,R3,R6; BEQ R5,R0,L; OR R7,R8,R9.

The LW to ADD edge is a true RAW dependence. In the five-stage timing model, it requires forwarding plus one load-use stall. The ADD to SUB edge is another true RAW dependence, but ALU forwarding can avoid an additional stall. The SUB to BEQ edge means that the branch depends on a just-produced comparison value, so forwarding or delayed branch resolution matters. Finally, the BEQ to OR edge is a control hazard: fetch may need prediction and a flush if the branch is taken or mispredicted.

A timing diagram is therefore a proof of the schedule. For each dependence, mark the producer stage, consumer stage, available bypass path, and every inserted bubble. This prevents the common mistake of saying “forwarding solves hazards” without checking when the value actually becomes available.

11.5.6 Choosing the Cheapest Correct Fix

If an ALU result is ready at the end of EX, its consumer enters EX in the next cycle, and an ALU bypass path already exists, the hazard unit should forward the result; no bubble is needed. If the producer is a load, the hazard unit must stall until the data is available and then forward it. The correct resolution depends on when the value becomes available, not only on the fact that a dependence exists.

11.6 Decision Procedure and Week 12 Bridge

For an unfamiliar parallel or pipeline problem, use the following procedure.

1. Identify the streams: count the instruction streams and data streams.
2. For a pipeline conflict, name the blocked stage and the requested resource or operand.

3. Classify the hazard as structural, data, or control.
4. Ask whether duplication, forwarding, scheduling, prediction, or a stall resolves it without changing program meaning.
5. Count the bubbles or flushes and update the timing diagram.
6. State the trade-off in throughput, latency, area, power, complexity, and recovery cost.

The central discipline is to connect a label to evidence. “Many ALUs” does not establish SIMD; a dependence does not by itself determine whether a stall is needed; and a faster clock does not by itself establish higher throughput. The organization, timing, and available resources must all be named.

Next week turns these mechanisms into worked instruction-pipelining problems and compares RISC and CISC choices that make hazards easier or harder to manage.

Exercises

1. Classify each workload as SISD, SIMD, MISD, or MIMD and justify the classification by naming its instruction and data streams: (a) one scalar core sums an array; (b) one vector instruction adds eight array elements; (c) four cores run independent matrix tiles.
2. For six independent instructions in a balanced five-stage pipeline, compute the sequential time, ideal pipelined time, and finite-stream speedup. State the limiting large-stream speedup.
3. Diagnose each hazard and give its cheapest correct response: (a) IF and MEM request one single-ported memory; (b) an ALU result is ready at the end of EX and its consumer enters EX next cycle; (c) a load’s data is available only after MEM.
4. For the running stream, identify every true dependence and the control hazard. State which dependence needs one load-use bubble and which can be handled by ALU forwarding.
5. A workload has branch frequency $f_{br} = 0.20$, misprediction rate $p_{miss} = 0.05$, and flush penalty $P_{flush} = 3$ cycles. Compute the approximate branch contribution to CPI and explain what happens if a deeper pipeline raises the penalty to 5 cycles.

Solutions

1. (a) SISD: one instruction stream operates on one scalar data stream. (b) SIMD: one instruction stream applies the same operation to many data elements. (c) MIMD: the cores run independent instruction streams over independent data regions.
2. Sequential time is $6 \times 5 = 30$ stage-cycles. The ideal pipelined time is $5 + (6 - 1) = 10$ cycles. The finite-stream speedup is $30/10 = 3$. For a large independent stream, the ideal speedup approaches 5, not 1, because one instruction can complete each cycle after fill.
3. (a) Structural hazard: split or duplicate the memory if the conflict justifies its area and power; otherwise insert a bubble. (b) Data/RAW hazard with a value available in time: forward the ALU result, with no bubble. (c) Data/RAW load-use hazard: stall until MEM produces the value, then forward it.
4. The LW produces $R1$, which ADD reads: true RAW and one load-use stall. ADD produces $R3$, which SUB reads: true RAW handled by ALU forwarding. SUB produces the comparison value used by BEQ: another true RAW whose handling depends on branch-resolution timing. BEQ changes the next instruction address, so the following OR is exposed to a control hazard and may be flushed after a wrong prediction.

5. The branch contribution is

$$\Delta\text{CPI} \approx 0.20 \times 0.05 \times 3 = 0.03.$$

If the penalty rises to 5 cycles, it becomes $0.20 \times 0.05 \times 5 = 0.05$ CPI. The clock may be faster, but each misprediction costs more cycles, so useful throughput depends on both effects.

References and Source Boundaries

The deck’s references are retained without changing their scope. Hamacher, Vranesic, Zaky, and Manjikian, *Computer Organization and Embedded Systems*, 6th ed., Chs. 5 and 10, is a chapter-level course anchor; the current repository index does not page-verify these chapters [1]. Stallings, *Computer Organization and Architecture: Designing for Performance*, 11th ed. (the repository key is retained as Stallings2015), is cited for S16.4, “Instruction Pipelining,” PDF pp. 667–685, and S20.1, “Multiple Processor Organizations,” PDF p. 847 [2]. Flynn’s taxonomy is not attributed to Stallings here: its treatment in the indexed 11th edition is brief, so the course anchor remains Hamacher Ch. 5. Page numbers are used only where the deck itself marks them as checkable.

References

- [1] V. Carl Hamacher, Zvonko G. Vranesic, Safwat G. Zaky, and Naraig Manjikian. *Computer Organization and Embedded Systems*. McGraw-Hill, 6th edition, 2012. ISBN 978-0-07-338065-0. Bib key retained as Hamacher2002_computer_organization for citation-key stability across existing lectures; the file indexed under master_supporting_docs/CS401/supporting_books/Hamacher2002/ and page-cited by Week 8 is this 6th edition, not the 5th ed. (2002) the key name suggests – see that folder’s index.md Edition notice, 2026-08-11.
- [2] William Stallings. *Computer Organization and Architecture: Designing for Performance*. Pearson, 10th edition, 2015.